Live in-call deepfake detection: implementation plan

How to detect during the call, not after it ends

---

1. The core idea: sliding window, not record-then-analyze

You cannot wait for the call to end before scoring it. Instead, as audio streams in from Asterisk, keep a rolling buffer and score it continuously while the call is still connected:

- Every roughly 1 second, take the **last 4 seconds** of buffered audio — the window RawTFNet expects — and run it through the model.
- This produces a fresh spoof score about once per second, throughout the call, instead of one score at the end.
- Maintain a running decision (e.g. three consecutive high-risk windows before raising an alert) so a single noisy window does not trigger a false alarm mid-call.

This is the same pattern used by live captioning or live translation systems: overlapping chunks processed continuously, rather than waiting for the whole recording.

2. What this requires from Asterisk

Asterisk needs to hand over audio while the call is active, not a recording afterward. Two realistic options:

- **AudioSocket** — Asterisk streams raw audio over a TCP connection to your app in real time, while the call is still live.
- **ARI (Asterisk REST Interface) with external media** — same idea, more control, slightly more setup work.

In either case, the detection process must be a long-running service that Asterisk feeds continuously, not a script run on a saved .wav file after hangup.

3. Latency budget

Being able to call this “live” depends on the total delay through the pipeline: audio capture → buffering the 4-second window → inference → decision. RawTFNet-16 is light (2.9G MACs), so inference itself should be fast even on a Raspberry Pi 4 — but running it inside the TrustZone enclave adds overhead that should be measured directly, since latency with vs without the TEE is a legitimate thesis result in its own right.

4. Showing the result during the call

Ranked roughly by how “live” they feel and how much extra work they require:

- **Dashboard or log that updates every ~1 second** — simplest to build; enough to demonstrate live detection for a thesis.
- **Push notification or alert the moment risk crosses a threshold** — better fits the “gateway” framing of flagging suspicious calls as they happen.

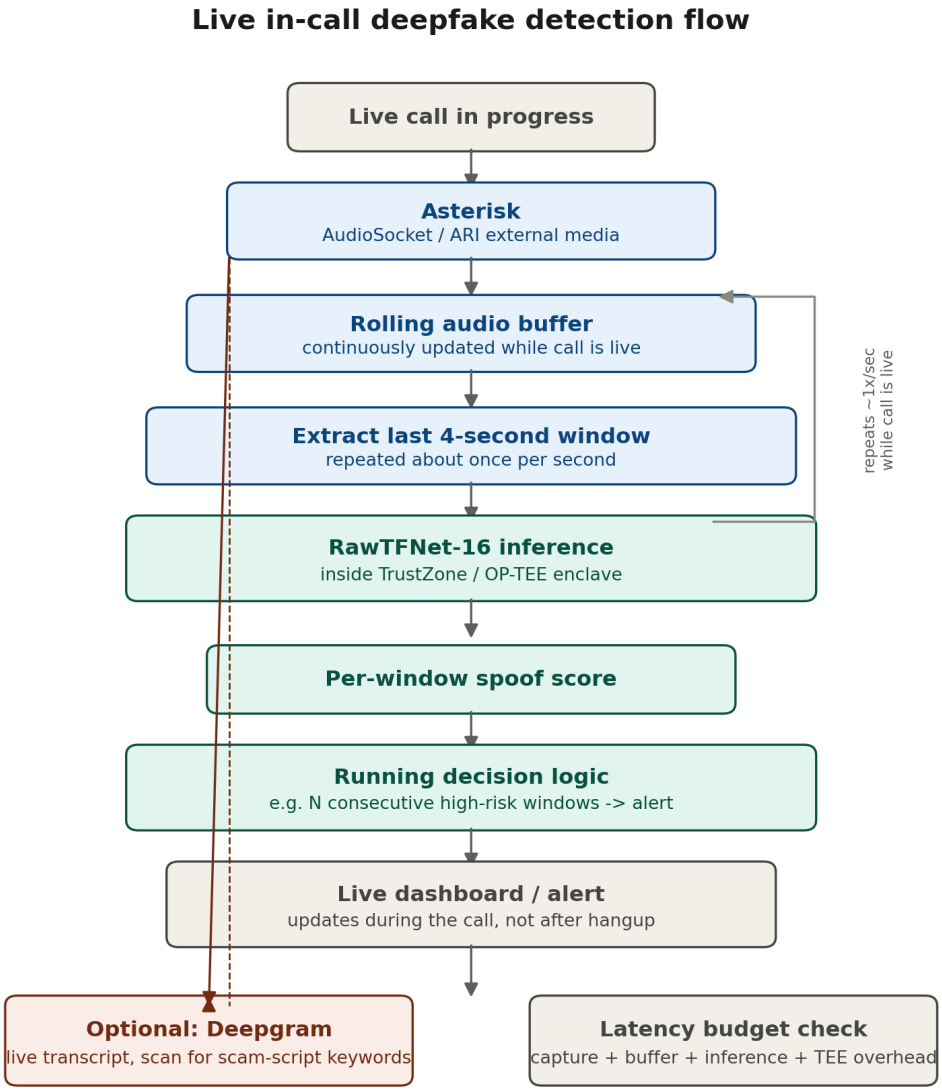
- **In-call audio cue** (warning tone or spoken message injected into the call) — most ambitious; requires Asterisk to inject audio back, not just capture it. Likely out of scope unless there is spare time.

Recommended target: the dashboard/alert version. It proves the “live” claim — a result appears while the call is still connected, with a measured latency number — without needing to build in-call audio injection.

5. Where Deepgram fits (optional, not core)

Deepgram does speech-to-text, not spoof detection — RawTFNet needs the raw waveform, and transcription throws away the acoustic artifacts it relies on. A live transcript branch is only useful as a separate, parallel feature: scanning for known scam-script keywords alongside (not instead of) the audio deepfake score.

Flowchart: live detection pipeline



End-to-end flow from live call audio to a dashboard result, with the optional Deepgram branch and the latency check called out separately.